1 PRELIMINARIES

1.1 Review of Wang et al. scheme

$Setup(1^\lambda) \rightarrow (PK, MSK)$: The setup algorithm $Setup$ uses the group generation algorithm $PairGen(1^\lambda)$ to initialize the system. It generates a description of the group and a multilinear map, denoted $\Gamma = (p, \mathbb{G}_0, \mathbb{G}_1, \mathbb{G}_2, e_0, e_1)$ where $\mathbb{G}_0$ has generator g_0. Random values α and β are selected along with a hash function $\mathbb{H} : \{0,1\}^* \rightarrow \mathbb{G}_0$. Then, it computes the values $A = e_0(g_0, g_0)^\alpha$ and $B = g_0^\beta$. The public key and master secret are defined as $PK = (\Gamma, g_0, A, B, H)$ and $MSK = (\alpha, \beta)$, respectively.

$KeyGen(MK, S) \rightarrow (SK)$: Given a attributes set S, produce a secret key SK. For $\forall x_i \in S$, r_i is randomly selected in $\mathbb{Z}_p^*$, and the sum $r = \sum_{i=1}^n r_i$ is computed. The component $\widetilde{D} = g^{(\alpha+r)/\beta}$ is then determined, together with another random element $r' \in \mathbb{Z}_p$. For each $x_i \in S$, compute $D_i = g_0^{r_i} H(x_i)^{r'}$, then calculate $\hat{D} = g_0^{r'}$ and $\check{D} = \prod_{i=1}^n H(x_i)^\beta$. The private key is $SK = (\widetilde{D}, \check{D}, \hat{D}, D_{i x_i \in S})$.

$Encrypt(PK, M, W, w) \rightarrow (CT, I_w)$: Let $M \in \mathbb{G}_1$ be the plaintext message, $W = (W_1, W_2, \ldots, W_n)$ denote an AND-gate access policy associated with a keyword w, choose randomly $s, s' \in \mathbb{Z}_p^*$, compute $C = MA^s, \widetilde{C} = B^s, \hat{C} = g_0^s$. For each attribute in W, it computes $C_i = H(W_i)^s$, and also defines $\check{C}_1 = e_0(B, g_0^{ws'})$ and $\check{C}_2 = g_0^{ss'}$. The ciphertext is $CT = (C, \tilde{C}, \hat{C}, C_i)$, and the index is $I_w = (C_i, \check{C}_1, \check{C}_2)$.

$Trapdoor(SK, w) \rightarrow (t_w)$: For a keyword w, the user U_i creates the trapdoor as follows.

$$t_w = e_0(\check{D}, g_0^w) \tag{1}$$

$Test(I_w, t_w) \rightarrow (1 \; or \; 0)$: The server checks whether index I_w and trapdoor t_w satisfy

$$e_1(t_w, \check{C}_2) \stackrel{?}{=} e_1(\prod_{i=1}^n C_i, \check{C}_1) \tag{2}$$

Output 1 when the above equation hold; otherwise, output 0.

$Decrypt(SK, CT) \rightarrow (M)$: For decryption, if the attribute set S meets the requirements of the policy W, such that $x_i = W_i$ holds for each $i \in [1, n]$, then the message M can be retrieved from CT by first executing the following calculation:

$$E = \prod_{i=1}^n \frac{e_0(D_i, \hat{C})}{e_0(\check{D}, C_i)} = e_0(g_0, g_0)^{rs} \tag{3}$$

Then, The plaintext message M is recovered by:

$$M = C/(e_0(\tilde{D}, \tilde{C})/E) \tag{4}$$

1.2 Security Assessment of Wang et al. Scheme

In the scheme [1], a CP-ABSE scheme is introduced, which aims to conceal the access policy. To enhance decryption efficiency, their approach incorporates a testing phase and an aggregation function during the decryption process. However, we have identified a significant flaw: the ciphertext components involved in both the key generation and encryption phases inadvertently disclose information about the associated access policy. Put differently, their scheme reveals information about the access policy. In the following, we will demonstrate why this scheme fails to effectively hide the access policy.

Suppose an adversary is aware of the set of attributes. In that case, the adversary can leverage specific components of the ciphertexts and public parameters to determine whether a guessed access policy is correct. Specifically, define the tuple $(g_0, C_i, \hat{C}, H(W_i^*))$ as an instance of the DDH problem. The adversary utilizes this tuple to perform a DDH test, aiming to determine whether the guessed access policy W_i^* correctly matches the actual policy W_i. The specific DDH test attack process is as follows.

$$e_0(\prod_{i=1}^{n} C_i, g_0) = e_0(\prod_{i=1}^{n} H(W_i^*), \hat{C}) \tag{5}$$

If the equation above is true, the adversary can deduce that $W^* = W$. In other words, the DDH test attack succeeds due to the ciphertext component C_i and $\hat{C}$, thus the scheme [1] fails to achieve hidden access policy.

2 Security analysis

Theorem 1. *Let $\xi_0, \xi_1, \xi_T, \mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ be specified as the generic bilinear group model. For any adversary $\mathcal{A}$ issuing up to q queries to the oracles for executing group operations in $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$, the bilinear map e, and interactions within the IND-sCP-CKA security game. We conclude that the advantage of adversary $\mathcal{A}$ in the game is $\mathcal{O}(q^2/p)$.*

Proof. Similar to [2], we examine a simulator $\mathcal{B}$ that engages in the following game with $\mathcal{A}$. The adversary $\mathcal{A}$ manages three distinct lists of pairs.

$$L_{\mathbb{G}_1} = \{\langle F_{1,l}, \xi_{1,l}\rangle : l = 1, \ldots, \tau_1\},$$

$$L_{\mathbb{G}_2} = \{\langle F_{2,l}, \xi_{2,l}\rangle : l = 1, \ldots, \tau_2\},$$

$$L_{\mathbb{G}_2} = \{\langle F_{T,l}, \xi_{T,l}\rangle : l = 1, \ldots, \tau_T\}$$

In this context, $F_{\tau,l}(\tau \in \{1, 2, T\})$ represent multivariate polynomials corresponding to $\mathcal{A}$'s queries. The $\xi_{T,l}(\tau \in \{1, 2, T\})$ are random strings in $\{0,1\}^*$ generated as the results for each query.

We set $F_{1,1} = 1, F_{2,1} = 1$ and $F_{T,1} = 1$, thereby mapping $\xi_{1,1}, \xi_{2,1}$ and $\xi_{T,1}$ to the string representations $\xi_1(1)$ of g_1, $\xi_2(1)$ of g_2 and $\xi_T(1)$ of $e(g_1, g_2)$,

respectively. In subsequent queries, adversary $\mathcal{A}$ interacts with simulator $\mathcal{B}$ using the ξ-representations of group elements. In the actual selective security game, the challenger selects random real values for each query and stores them in lists. Conversely, simulator $\mathcal{B}$ maintains multivariate polynomials F in its lists instead of selecting real values. Ultimately, $\mathcal{B}$ provides all query tuples to $\mathcal{A}$ for verification of the game's consistency. Whenever $\mathcal{A}$ submits queries, $\mathcal{B}$ updates its lists and returns corresponding new random strings to $\mathcal{A}$. The detailed oracle queries of $\mathcal{A}$ are outlined as follows:

Group operation. For two operands $\xi_\tau(x)$ and $\xi_\tau(y)$, where $x, y \in \mathbb{Z}_p^*$ and $\tau \in \{1, 2, T\}$, if $\xi_\tau(x)$ or $\xi_\tau(y)$ is absent from $L_{\mathbb{G}_\tau}$, return $\perp$. Otherwise, $\mathcal{B}$ computes $F = x + y \bmod p$ and verifies if F exists in $L_{\mathbb{G}_\tau}$. If present, $\mathcal{B}$ returns $\xi_\tau(F)$; if not, $\mathcal{B}$ assigns $\xi_\tau(F)$ a random string in $\{0,1\}^*$ distinct from all strings in L_{G_τ}. Then, $\mathcal{B}$ appends $(F, \xi_\tau(F))$ to $L_{\mathbb{G}_\tau}$ and responds to $\mathcal{A}$ with $\xi_\tau(F)$.

For a given string $\xi_2(x)$, if $\xi_2(x)$ is not present in list $L_{\mathbb{G}_2}$, return $\perp$. Otherwise, if x is already in $L_{\mathbb{G}_1}$, $\mathcal{B}$ returns $\xi_1(x)$ to $\mathcal{A}$; if not, $\mathcal{B}$ assigns $\xi_1(x)$ a unique random string in $0, 1^*$ distinct from all strings in $L_{\mathbb{G}_1}$. Then, $\mathcal{B}$ adds $\langle F, \xi_T(F)\rangle$ to $L_{\mathbb{G}_1}$ and responds to $\mathcal{A}$ with $\xi_1(x)$.

For two operands $\xi_1(x)$ and $\xi_2(y)$, if $\xi_1(x)$ is not in $L_{\mathbb{G}_1}$ or $\xi_2(y)$ is not in $L_{\mathbb{G}_2}$, return $\perp$. Otherwise, $\mathcal{B}$ calculates $F = xy \bmod p$ and checks if F exists in $L_{\mathbb{G}_T}$. If present, $\mathcal{B}$ returns $\xi_T(F)$; if not, $\mathcal{B}$ assigns $\xi_T(F)$ a unique random string in $\{0,1\}^*$ distinct from all strings in $L_{\mathbb{G}_T}$. Then, $\mathcal{B}$ appends $(F, \xi_T(F))$ to $L_{\mathbb{G}_T}$ and responds to $\mathcal{A}$ with $\xi_T(F)$.

Using the aforementioned group operations, simulator $\mathcal{B}$ executes the selective security game as follows.

Setup. Adversary $\mathcal{A}$ selects two distinct access control policies, P_0 and P_1, where $P_i = \{P_{i,1}, P_{i,2}, ..., P_{i,n}\}$ for $i \in \{0,1\}$, and transmits them to $\mathcal{B}$. Instead of assigning real values to the master secret key variables $(\alpha, b, \{\{y_{i,t_i}\}_{1 \leqslant t_i \leqslant n_i}\}_{i \in [1,n]})$, $\mathcal{B}$ maintains their corresponding multivariate polynomials in the lists. Subsequently, $\mathcal{B}$ updates the lists by incorporating tuples for each component of the public parameters $(e(g_1, g_2)^\alpha, g_1^b, \{\{g_1^{y_{i,t_i}}\}_{1 \leqslant t_i \leqslant n_i}\}_{i \in [1,n]})$. Finally, $\mathcal{B}$ delivers the new random strings from the updated lists to $\mathcal{A}$.

Phase 1. $\mathcal{A}$ selects an attribute list $S = \{x_1, x_2, ..., x_n\} = \{v_{1,t_1}, v_{2,t_2}, \ldots, v_{n,t_n}\}$ for querying $O_{Keygen}(S)$ and $O_{Trapdoor}(S, W')$ as described below:

- $\mathcal{O}_{keygen(S)}$: Initially, $\mathcal{B}$ appends the new tuple $\langle \alpha x_u, \xi_T(\alpha x_u)\rangle$, corresponding to the element $e(g_1, g_2)^{\alpha x_u}$ with the freshly introduced variable x_u, to the list $L_{\mathbb{G}_T}$. Following this, $\mathcal{B}$ proceeds to update the relevant lists by inserting new tuples related to the private key $(x_u, g_2^{\frac{\alpha+\beta}{b}}, g_2^{\beta + yi, t_i \lambda_i}, {g_2^{\lambda_i}}_{i \in [1,n]})$, where β and each λ_i denote newly defined variables.
- $\mathcal{O}_{Trapdoor(S,W')}$: $\mathcal{B}$ executes $\mathcal{O}_{Keygen}(S)$ and subsequently updates the lists by adding new tuples associated with the trapdoor $(x_u + s, g_2^{\frac{\alpha+\beta}{b} s \sum_{i=1}^{n} H(w_j')}$, $\{g_2^{s(\beta + y_{i,t_i}\lambda_i)}, g_2^{s\lambda_i}\}_{i \in [1,n]})$, where s represents a fresh variable.

Challenge. $\mathcal{A}$ chooses two distinct keyword sets, denoted as W_0 and W_1, and submits the tuples (W_0, P_0) and (W_1, P_1). The challenger, operating within the

real selective security scheme, randomly selects a value $\sigma \in \mathbb{Z}_p^*$ to encrypt the keyword set W_b using the corresponding public key P_b, where $b \in \{0,1\}$. Meanwhile, $\mathcal{B}$ constructs a challenge ciphertext, represented as $(\widetilde{C}, C^*, C_1, \{C_i, \hat{C}_i\}_{i\in[1,n]})$, according to the following procedure:

- For the component $\widetilde{C}$, $\mathcal{B}$ adds tuples of the form $(\alpha r, \xi_T(\alpha r))$ to the list $L_{\mathbb{G}_T}$, where r represents a freshly chosen variable. For the set $\{C_i\}_{1\leq i\leq n}$, $\mathcal{B}$ adds tuples $(r_i, \xi_1(r_i))$ in the list $L_{\mathbb{G}_1}$, utilizing new variables r_i that satisfy the condition $r = \sum_{i=1}^{n} r_i$.
- For the component C_1, if the keywords W_0 and W_1 are identical, $\mathcal{B}$ incorporates a tuple $(br/\sum_{i=1}^{m} H(w_{0,j}), \xi_1(br/\sum_{i=1}^{m} H(w_{0,j})))$ into the list $L_{\mathbb{G}_1}$. Otherwise, $\mathcal{B}$ adds a tuple $(\theta_1, \xi(\theta_1))$, where θ_1 is a newly selected variable, to the list $L_{\mathbb{G}_1}$.
- For the components $\{\hat{C}_i\}_{1\leq i\leq n}$, if $y_{i,t_i} \in P_{0,i}$ and $y_{i,t_i} \in P_{1,i}$, $\mathcal{B}$ adds a tuple $(y_{i,t_i} r_i, \xi_1(y_{i,t_i} r_i))$ into the list $L_{\mathbb{G}_1}$. If $y_{i,t_i} \notin P_{0,i}$ and $y_{i,t_i} \notin P_{1,i}$, $\mathcal{B}$ adds a tuple $(r_{i,t_i}, \xi_1(r_{i,t_i}))$ to list $L_{\mathbb{G}_1}$, where r_{i,t_i} is a newly generated variable.If $y_{i,t_i} \notin P_{0,i}$ and $y_{i,t_i} \in P_{1,i}$, or $y_{i,t_i} \in P_{0,i}$ and $y_{i,t_i} \notin P_{1,i}$, $\mathcal{B}$ adds a tuple $(\theta_2, \xi_1(\theta_2))$, with θ_2 as a freshly selected variable, in the list $L_{\mathbb{G}_1}$.

Phase 2. $\mathcal{A}$ continues to issue queries in the same manner as in Phase 1. With the additional constraints that When the keywords W_0 and W_1 are distinct, $\mathcal{A}$ is prohibited from querying $\mathcal{O}_{Keygen}(S)$ and $\mathcal{O}_{Trapdoor}(S, W)$ if the attribute list S satisfies both policies P_0 andP_1.

After no more than q oracle queries, $\mathcal{A}$ outputs $\sigma' \in \{0,1\}$. $\mathcal{B}$ then selects $\sigma \in \{0,1\}$ randomly and forms the challenge ciphertext by replacing $g_1^{\theta_1}$ with $g_1^{br/\sum_{j=1}^{m} H(w_{\sigma,j})}$ and $g_1^{\theta_2}$ with $g_1^{y_{i,t_i} r_i}$ in $L_{\mathbb{G}_1}$, using random $\mathbb{Z}_p^*$ values. Finally, $\mathcal{B}$ delivers the updated list tuples to $\mathcal{A}$.

Subsequently, we present an in-depth analysis of $\mathcal{B}$'s simulation. We assert that $\mathcal{B}$'s simulation remains flawless provided no "unanticipated collision" occurs, meaning that the random strings assigned to variables must not yield a zero difference for any pair of query polynomials $F_{\tau,l}$ and $F_{\tau,l'}$ (where $\tau \in \{1,2,T\}$) for specific l, l'. Thus, an "unanticipated collision" arises only when, for any pair of queries (within a list $L_{\mathbb{G}_1}$, where $\tau \in \{1,2,T\}$ linked to two distinct non-zero polynomials $F_{\tau,l}$ and $F_{\tau,l'}$, the condition $F_{\tau,l} - F_{\tau,l'} = 0$ holds for certain l, l'. We demonstrate the emergence of such an "unanticipated collision" through the following two scenarios:

Before substitution. According to the Schwartz-Zippel lemma [3, 4] the likelihood of an "unanticipated collision" occurring within $L_{\mathbb{G}_1}$, $L_{\mathbb{G}_2}$, and $L_{\mathbb{G}_T}$ is bounded above by $\mathcal{O}(q^2/p)$. Further information on this lemma is available in references [3, 4].

After substitution. We demonstrate that no new equivalences arise between polynomials $F_{k,l}$ and $F_{k,l'}$ even when $\mathcal{B}$ replaces θ_1 with $br/\sum_{j=1}^{m} H(w_{\sigma,j})$ and θ_2 with $y_{i,t_i} r_i$ at the simulation's conclusion. In other words, we need to prove that adversary $\mathcal{A}$ cannot formulate a query for a non-zero $F = F_{k,l} - F_{k,l'}$ such that $F = 0$ after variable substitution. Clearly, in the selective security game, for two distinct keyword queries W_0 and W_1, $\mathcal{A}$ seeks to differentiate

$g_1^{br/\sum_{j=1}^{m} H(w_{0,j})}$ from $g_1^{br/\sum_{j=1}^{m} H(w_{1,j})}$. Given $\delta_1 \in \mathbb{Z}_p^*$, the probability of distinguishing $g_1^{br/\sum_{j=1}^{m} H(w_{0,j})}$ from $g_1^{\delta_1}$ equals half the probability of distinguishing $g_1^{br/\sum_{j=1}^{m} H(w_{0,j})}$ from $g_1^{br/\sum_{j=1}^{m} H(w_{1,j})}$. Consequently, we adjust the game so that if $\mathcal{A}$ can generate queries for $e(g_1, g_2)^{\gamma br}$ for some g_2^{γ}, it can distinguish $g_1^{\delta_1}$ from $g_1^{br/\sum_{j=1}^{m} H(w_{0,j})}$. Similarly, as noted above, if $\mathcal{A}$ can produce queries for $e(g_1, g_2)^{\gamma' y_{i,t_i} r_i}$ for some $g_2^{\gamma'}$, it can distinguish $g_1^{\delta_2}$ from $g_1^{y_{i,t_i} r_i}$. According to reference [5], we establish that $\mathcal{A}$ cannot effectively construct queries for $e(g_1, g_2)^{\gamma br}$ or $e(g_1, g_2)^{\gamma' y_{i,t_i} r_i}$, thereby ensuring the indistinguishability of keywords and access control policies.

Case 1. To derive the term br, it is observed that the only available information about br comes from the expression $br/\sum_{j=1}^{m} H(w_{\sigma,j})$. According to the simulation, when $\mathcal{B}$ replaces θ_1 with this value, it cannot generate a valid search token under the condition $S = P_0 \wedge S = P_1$, due to the mismatch between w_0 and w_1. Therefore, even if $\mathcal{B}$ uses the correct value of $y_{i,t_i} r_i$ for θ_2, it still lacks the capability to carry out the keyword search. As a result, $\mathcal{A}$ gains no advantage in recovering the form of br or constructing a query involving $e(g_1, g_2)^{\gamma br}$.

Case 2. Consider any $y_{i,t_i} r_i$ that emerges following $\mathcal{B}$'s substitution. We posit that $\mathcal{A}$ can formulate a query for $e(g_1, g_2)^{\nu}$, where ν is a non-zero polynomial containing θ_2 and evaluates to zero after $\mathcal{B}$ replaces θ_2 with $y_{i,t_i} r_i$ (Additionally, the rest of the variables in ν are replaced.).

To construct such ν, adversary $\mathcal{A}$ must eliminate of y_{i,t_i} within ν. It is known that the access structure may include a distinct attribute value $v_{i,t_i'}$ (where $t_i' \neq t_i$) for the same attribute P_i. The adversary $\mathcal{A}$ may also obtain ciphertexts of the form $g_1^{y_{i,t_i'} r_i}$ corresponding to $v_{i,t_i'}$. Consequently, $\mathcal{A}$ has two potential methods to derive the term $y_{i,t_i} r_i$: one by computing the pairing of $g_1^{r_i}$ with $g_2^{\beta + y_{i,t_i} \lambda_i}$, and the other by pairing $g_1^{y_{i,t_i'} r_i}$ with $g_2^{\beta + y_{i,t_i} \lambda_i}$.

- When $\mathcal{A}$ pairs $g_1^{y_{i,t_i'} r_i}$ with $g_2^{\beta + y_{i,ti} \lambda_i}$, $\mathcal{A}$ must obtain information about $g_2^{y_{i,t_i'} \lambda_i}$ to pair it with $g_1^{\theta_2}$. However, this is infeasible because $g_2^{y_{i,t_i'} \lambda_i}$ remains unavailable throughout the simulation.
- If $\mathcal{A}$ pairs $g_1^{r_i}$ with $g_2^{\beta + y_{i,ti} \lambda_i}$, $\mathcal{A}$ can generate a query containing the term $\beta r_i + y_{i,t_i} r_i \lambda_i$. To form the term $\gamma' y_{i,t_i} r_i$, let $\gamma' = \lambda_i \gamma''$ for some λ''; thus, $\mathcal{A}$ must first construct the term $\lambda'' \beta r_i$. Given the relation $r = \sum_{i=1}^{n} r_i$, $\mathcal{A}$ needs to derive the term $\gamma'' \beta r$. Based on the queries available in the simulation, the only method to construct $\gamma'' \beta r$ is by combining $g_2^{\frac{\alpha+\beta}{b} \sum_{j=1}^{m} H(w_j) s}$ with $X^{\frac{r}{\sum_{j=1}^{m} H(w_j)}}$ to obtain the query term $\alpha r s + \beta r s$. To isolate $\beta r s$, $\mathcal{A}$ pairs $e(g_1, g_2)^{\alpha r}$ with $x_u + s$, yielding the term $\alpha r (x_u + s)$, and then cancels $\alpha r x_u$ using the term $-x_u \alpha r$. Once $\mathcal{A}$ obtains $\beta r s$, let $\gamma'' = \gamma''' s$ for some γ'''. Consequently, $\mathcal{A}$ could derive $\gamma'' \beta r i$ by constructing a query of the form $\gamma''' s(\beta(r \sum_{i' \neq i} r_{i'}))$. However, constructing such a query is infeasible for $\mathcal{A}$. The analysis is as follows.

As the secret s is randomly selected by the user and remains unknown to $\mathcal{A}$, $\mathcal{A}$ cannot identify a γ''' such that $\gamma'' = \gamma'''s$. Consequently, $\mathcal{A}$ is unable to generate γ''' while meeting the above condition.

Based on the simulation of the challenge ciphertext $\hat{C}_i$, it is evident that $v_{i,t_i} \notin P_{b,i} \wedge v_{i,t_i} \in P_{1,i}$. In other words, $\mathcal{A}$ cannot obtain the secret key SK or the search token $Token$ to perform search operations. As noted in [6], there must be at least one unknown r'_i, preventing $\mathcal{A}$ from acquiring the component $\sum_{i' \neq i} r'_i$. Consequently, $\mathcal{A}$ cannot formulate a query of the form $\gamma'''s(\beta(r\sum_{i' \neq i} r'_i))$.

Theorem 2. *The HCP-ABMKSE scheme achieves security in the IND-sCP-CPA model, provided no polynomial-time adversary $\mathcal{A}$ can succeed in the aforementioned game with a non-negligible advantage ϵ .*

Proof. Suppose there exists a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ capable of distinguishing between access structures P_0 and P_1 with a non-negligible advantage Λ. We construct a simulator $\mathcal{B}$ that can solve the DBDH problem with a non-negligible advantage of $\frac{\Lambda}{2}$. Using the parameters of a bilinear map $(e, p, g_1, g_2, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$, the challenger initially selects $(a', b', c') \in \mathbb{Z}_p^*$, $\gamma \in \{0, 1\}$, and $z \in \mathbb{G}_T$. Subsequently, $\mathcal{A}$ assigns $Z = e(g_1, g_2)^{a'b'c'}$ when $\gamma = 0$; otherwise, $Z = e(g_1, g_2)^z$. Then, $\mathcal{A}$ transmits the tuple $(g, g^{a'}, g^{b'}, g^{c'}, Z)$ to $\mathcal{B}$. Upon receiving the ciphertext M, the data owner produces the associated ciphertext $CW = \{\overline{C}, \widetilde{C}, C^*, C_1, \{C_i, \hat{C}_i\}_{i\in[1,n]}\}$. The IND-sCP-CPA security game between $\mathcal{A}$ and $\mathcal{B}$ proceeds as described below:

Initial. $\mathcal{A}$ submits two challenge access structures P_0 and P_1 to $\mathcal{B}$.

$\mathcal{B}$ selects two access structures P_0 and P_1 as the challenge for adversary $\mathcal{A}$, and randomly chooses $\gamma \in \{0, 1\}$. $\mathcal{A}$ submits a challenge access policy P^* and a keyword set W to $\mathcal{B}$.

Setup. $\mathcal{B}$ selects random $\alpha' \in \mathbb{Z}_p^*$ and sets $\alpha = a'b'$, then calculates $A = e(g_1, g_2)^{\alpha} = e(g_1, g_2)^{a'b'}$ and sets $X = g_1^b = g_1^{b'}$. The hash function $H : \{0, 1\}^* \to \mathbb{Z}_p^*$ is defined. $\mathcal{B}$ sends (A, X) to $\mathcal{A}$.

Phase 1. $\mathcal{A}$ makes private key queries to $\mathcal{B}$ for an attribute set $S = (x_1, x_2, \ldots, x_n)$, where S does not satisfy the access structures P_0 or P_1. $\mathcal{B}$ runs the $Keygen$ algorithm to generate the private key SK. $\mathcal{B}$ randomly selects $\beta \in \mathbb{Z}_p^*$ and defines $\beta = \beta^* - a'$. Next, $\mathcal{B}$ computes $K_0 = g_2^{\frac{\alpha+\beta}{b}} = g_2^{\frac{\alpha+\beta^*-a'}{b}}$. It then selects $x_u, \lambda_i \in \mathbb{Z}_p^*$ (for $i \in [1, n]$), and computes $B = A^{x_u} = e(g_1, g_2)^{\alpha x_u}$, $K'_i = g_2^{\beta^*-a'+H(x_i)\lambda_i}$, $K''_i = g_2^{\lambda_i}$. Subsequently, it outputs $SK = \{x_u, K_0, \{K'_i, K''_i\}_{i\in[1,n]}\}$. Finally $\mathcal{B}$ sends SK to $\mathcal{A}$.

Challenge. $\mathcal{A}$ submits two equal-length messages $M_0, M_1 \in \mathbb{G}_T$ and a keyword set W. $\mathcal{B}$ randomly selects $\gamma \in \{0, 1\}$ and generates the ciphertext $CW^* = \{\widetilde{C}_\gamma, C^*_\gamma\}$ for message M_γ under access policy P_γ, where $C^* = g_1^{bu_\gamma} = g_1^{bc'}$, $\widetilde{C}_\gamma = M_\gamma \cdot A^{u_\gamma} = M_\gamma \cdot e(g_1, g_2)^{\alpha u_\gamma} = M_\gamma \cdot e(g_1, g_2)^{a'b'c'} = M_\gamma \cdot Z$. Finally $\mathcal{B}$ sends CW^* to $\mathcal{A}$.

Phase 2. Same as Phase 1, with the same restrictions.

Guess. $\mathcal{A}$ outputs a guess $\gamma' \in \{0, 1\}$ for γ. It wins if $\gamma' = \gamma$.

Since S does not satisfy P_0 or P_1, $\mathcal{A}$ cannot decrypt CW or extract information about the access structure P_γ. We analyze two cases.

- Case $\gamma = 0$: $\mathcal{C}$ receives $t_0 = (g_1^{a'}, g_1^{b'}, g_1^{c'}, Z = e(g_1, g_2)^{a'b'c'})$. Set $b' = b$, $a' = \alpha/b'$, and $c' = u_\gamma$. Then, $X = g_1^{b'}$, $C_\gamma^* = g_1^{bu_\gamma} = g_1^{b'c'}$, $\overline{C} = M_0 \cdot e(g_1, g_2)^{\alpha u_\gamma} = M_0 \cdot e(g_1, g_2)^{a'b'c'}$, and the ciphertext CW is a valid encryption of M_0 under P_0. The probability that $\mathcal{A}$ correctly guesses $\gamma' = \gamma = 0$ is $\frac{1}{2} + \Lambda$.
- Case $\gamma = 1$: $\mathcal{C}$ receives $t_1 = (g_1^{a'}, g_1^{b'}, g_1^{c'}, e(g_1, g_2)^z)$. Set $b' = b$, $a' = \alpha/b'$, and $c' = u_\gamma$, and use $Z = e(g_1, g_2)^z$. Then, $\overline{C} = M_1 \cdot e(g_1, g_2)^z$, and the ciphertext CW appears random to $\mathcal{A}$ since z is independent of α. Thus, $\mathcal{A}$'s guess $\gamma' = \gamma = 1$ has probability $\frac{1}{2}$.

The advantage of $\mathcal{B}$ in solving the DBDH problem is:

$$\begin{aligned} &\left| \frac{1}{2} \Pr[\gamma' = \gamma \mid \gamma = 0] + \frac{1}{2} \Pr[\gamma' = \gamma \mid \gamma = 1] - \frac{1}{2} \right| \\ =&\left| \frac{1}{2} \left(\frac{1}{2} + \Lambda \right) + \frac{1}{2} \cdot \frac{1}{2} - \frac{1}{2} \right| = \frac{\Lambda}{2} \end{aligned} \tag{6}$$

Thus, if $\mathcal{A}$ has a non-negligible advantage Λ, $\mathcal{B}$ can solve the DBDH problem with non-negligible advantage $\frac{\Lambda}{2}$, contradicting the DBDH assumption. Hence, the HCP-ABMKSE scheme is IND-sCP-CPA secure.

References

1. Haijiang Wang, Xiaolei Dong, and Zhenfu Cao. Multi-value-independent ciphertext-policy attribute based encryption with fast keyword search. *IEEE Trans. Serv. Comput.*, 13(6):1142–1151, 2020.
2. Takashi Nishide. Cryptographic schemes with minimum disclosure of private information in attribute-based encryption and multiparty computation. *Ph. D. Thesis, The University of Electro-Communications*, 2008.
3. Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.
4. Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation, EUROSAM, France, 1979, Proceedings*, pages 216–226. Springer, 1979.
5. John Bethencourt, Amit Sahai, and Brent Waters. Ciphertext-policy attribute-based encryption. In *2007 IEEE Symposium on Security and Privacy (S&P 2007), 2007, Oakland*, pages 321–334. IEEE Computer Society, 2007.
6. Qingji Zheng, Shouhuai Xu, and Giuseppe Ateniese. VABKS: verifiable attribute-based keyword search over outsourced encrypted data. In *IEEE INFOCOM'14*, pages 522–530, 2014.